

Projet 7Wonders

Groupe A

Boutrik Alexandre, **Diallo** Elhadj Oumar,
Ghilane El Hassani Abdelouarite, **Landoulsi** Aziz,
Maniliuc David, **Rahmouni** Mohamed Khalil

11 décembre 2025

Université Côte d'Azur

Table des matières

1. Quelles fonctionnalités avons-nous réalisées ?
2. Quels grands choix de conception avons-nous faits ?
3. Comment avons-nous organisé les tests ?
4. Quels sont les points forts et les points faibles de notre implémentation ?
5. Comment avons-nous géré le projet ?
6. Quelles inflexions ont été prises en compte ?

**Quelles fonctionnalités
avons-nous réalisées ?**

Quelles fonctionnalités avons-nous réalisées ?

- Jeu complet avec *toutes les mécaniques* implémentées dont le chaînage et les ressources à choix
- Intégralité des effets des cartes et des merveilles
- Configuration utilisateur en toml
- Modes d'exécution :
 - **un** avec un déroulé visible et compréhensible (+log)
 - **plusieurs** pour un usage statistique
- Chargement des données de jeu a partir de fichiers json.
- Robots implémentés :
 - 1 robot choisissant des cartes au *hasard*
 - 4 robots *spécialistes* se concentrant sur un seul but
 - 2 robots *intelligents* choisissant les cartes en fonction du cours de la partie
 - 1 robot « *meta* » se servant des stratégies spécialistes
- 3 inflexions sur les 4 ont été implémentées

**Quels grands choix de
conception avons-nous faits ?**

Conception clé : Structure du projet

Architecture des modules :

Notre projet est structuré autour de plusieurs modules ayant pour rôle :

- **models** : Contient les objets du domaine représentant l'état du jeu (joueurs, cartes, merveilles, etc.).
- **core** : Abrite le `GameEngine`, le moteur principal qui orchestre le déroulement d'une partie.
- **ai** : Représente la logique et l'api des robots.
- **services** : Est garant de la construction et du calcul de score.
- **io** : Gère les entrées/sorties, notamment l'affichage des informations pour l'utilisateur via `ConsoleReporter`.
- **stats** : Responsable de l'analyse des statistiques des parties jouées.
- **utils** : Fournit des fonctionnalités transverses comme la gestion de la configuration (`config.toml`) et la journalisation.

Conception clé : Utilisation des design patterns

Design patterns :

Utilisation de *singleton*, *strategy* et *composite*

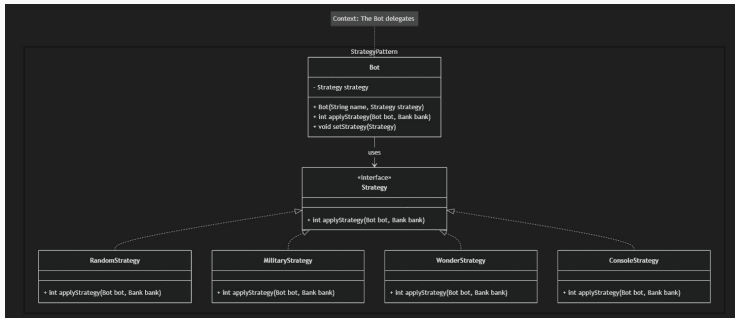


Singleton Pattern

Conception clé : Utilisation des design patterns

Design patterns :

Utilisation de *singleton*, *strategy* et *composite*

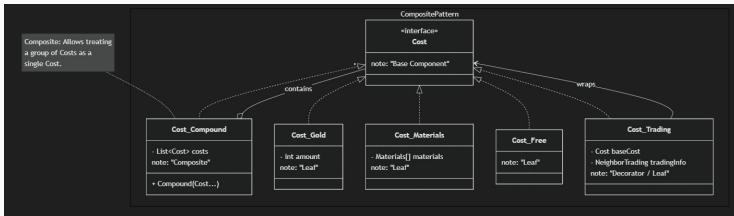


Strategy Pattern

Conception clé : Utilisation des design patterns

Design patterns :

Utilisation de *singleton*, *strategy* et *composite*



Composite Pattern

Conception clé : Modélisation des effets de jeu

Problématique

Comment représenter les 8 effets de jeu (victoire, ressources, actions spéciales...) de manière **flexible, sûre et maintenable** ?

Solution : Architecture modulaire avec Java moderne

- sealed interface

Effect

- Définit tous les types d'effets possibles
- Garantie de sécurité à la compilation

- Classes record
immuables

- Gold, Military, Science...
- Définition concise et sécurisée

- Méthode apply(Player player)

- Logique d'application spécifique
- Cohésion forte, couplage faible

- Avantages

- Ajout/modification d'effets sans risque
- Code clair et extensible

Comment avons-nous organisé les tests ?

Comment avons-nous organisé les tests ?

```
src/test/java/fr/uca/miage/sevenwonders
├── ai
│   ├── BotIT.java
│   ├── BotTest.java
│   └── strategies
│       ├── AnyResourceStrategyIT.java
│       ├── BlueStrategyIT.java
│       ├── EconomicStrategyIT.java
│       ├── MarStrategyIT.java
│       ├── MilitaryStrategyIT.java
│       ├── MinMaxStrategyIT.java
│       ├── RandomStrategyIT.java
│       ├── ScienceStrategyIT.java
│       └── WonderStrategyIT.java
├── core
│   ├── GameEngineIT.java
│   ├── GameEngineTest.java
│   └── session
│       ├── ActionExecutorTest.java
│       ├── ConflictManagerTest.java
│       ├── SessionInitializerTest.java
│       ├── TransactionManagerTest.java
│       └── TurnManagerTest.java
│   ├── SessionIT.java
│   └── SessionTest.java
├── models
│   ├── BankIT.java
│   ├── BankTest.java
│   ├── card
│   │   └── CardTest.java
│   ├── CostIT.java
│   ├── CostTest.java
│   ├── DeckIT.java
│   ├── DeckTest.java
│   ├── EffectIT.java
│   ├── NeighborTradingIT.java
│   ├── NeighborTradingTest.java
│   ├── player
│   │   ├── PlayerBoardTest.java
│   │   ├── PlayerMilitaryTest.java
│   │   ├── PlayerResourcesTest.java
│   │   ├── PlayerScienceTest.java
│   │   └── PlayerTest.java
│   └── wonder
│       ├── WonderStageTest.java
│       └── WonderTest.java
├── services
│   ├── ConstructionServiceTest.java
│   └── ScoreCalculatorTest.java
└── utils
    └── DeserializerIT.java

11 directories, 40 files
```

Stratégie de test :

*Tests unitaires (*Test.java) :*

- Couverture des composants critiques
- Isolation des fonctionnalités

*Tests d'intégration (*IT.java) :*

- Validation des interactions
- Tests des interfaces

Quels sont les points forts et les points faibles de notre implémentation ?

Points forts et points faibles

Points forts

- **Documentation robuste** : Javadoc, commentaires, rapports détaillés et diagrammes UML.
- **Expérience utilisateur** : Paramétrage fin du comportement du jeu via `config.toml`.
- **Affichage** : Résultats sous forme de tableaux clairs et interface interactive attrayante.

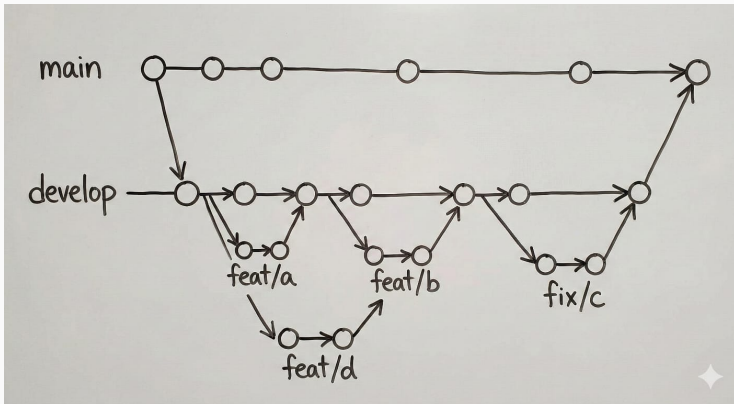
Points faibles

- **Logs peu lisibles** : Structure peu claire et messages insuffisamment explicites.
- **Architecture monolithique** : Choix initial pour sa simplicité, mais limitant pour :
 - **Évolutivité** : Ajout de fonctionnalités plus complexe
 - **Maintenance** : Dépendances fortes entre composants
- Utilisation de « *valeurs magiques* » pour les choix des robots et joueurs humains

**Comment avons-nous géré le
projet ?**

Comment avons-nous géré le projet ?

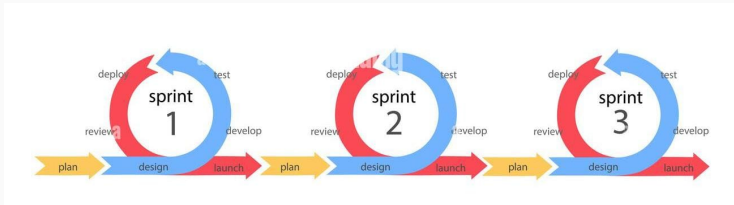
Gestion du code source :



Organisation des branches git

Comment avons-nous géré le projet ?

Organisation du temps de travail :



Division du travail en sprints agiles

Comment avons-nous géré le projet ?

Jalons non relayés :

- Traduction du jeu (i18n/l10n)
- Passage de l'architecture monolithique vers des micro-services
- Création d'un robot utilisant un *LLM* pour sa prise de décisions

**Quelles inflexions ont été prises
en compte ?**

Quelles inflexions ont été prises en compte ?

Inflexions implémentées

- **Inflexion 1 : Amélioration de la lisibilité** L'affiche utilise désormais des couleurs configurables par l'utilisateur dans la section `[colors]` du fichier `config.toml`.
- **Inflexion 2 : Lancement de parties en parallèle** Le nombre de threads est configurable via le paramètre `number_of_threads = 4` dans la configuration.
- **Inflexion 4 : Participation de l'utilisateur** L'utilisateur peut participer au jeu en activant l'option `enable_real_player = true` (ou `false` pour le mode automatique).

Inflexion non implémentée

- **Inflexion 3 : Mode duel** Cette fonctionnalité nécessitait des modifications trop importantes des règles de jeu et n'a pas pu être réalisée dans le délai imparti.

Questions ?

Merci pour votre attention